



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

Subject : SECP3133 High-Performance Data Processing

Semester : 2/2526

Section : 02

Task : Lab 2

Title : Real-Time Sentiment Analysis of Malaysian
Telecommunication App Reviews

Lecturer : Dr. Seah Choon Sen

Prepared by:

No.	Name	Matric Number
1	Lee Yin Shen	A23CS0236
2	Tan Zhi Ming	A23CS0189
3	Brendan Chia Yan Fei	A23CS0211
4	Choh Jing Yi	A23CS0296

Abstract

This report presents the design and implementation of a real-time sentiment analysis pipeline for Malaysian telecommunication app reviews. The project uses a Kaggle-based Google Play review dataset containing customer feedback for Malaysian telco providers. The raw review text was cleaned, labelled using rating-based sentiment rules, and used to train two machine learning models: Naive Bayes and Logistic Regression. Logistic Regression achieved the better accuracy at 86.46%, so it was selected as the deployed model for the streaming pipeline.

For the real-time component, Apache Kafka was used to stream review messages, while Apache Spark Structured Streaming consumed the messages, applied the TF-IDF vectorizer and trained Logistic Regression model, and wrote the prediction output into Elasticsearch. Kibana was then used to visualize the stored sentiment results through dashboards covering sentiment distribution, provider comparison, review trends, rating distribution, and negative-provider analysis. The system demonstrates how batch model development and streaming data processing can be integrated into a complete high-performance data processing workflow.

Table of Contents

Abstract.....	1
1. Introduction.....	4
1.1 Background	4
1.2 Problem Statement	4
1.3 Objectives.....	4
1.4 Scope	5
2. Data Acquisition and Preprocessing.....	5
2.1 Dataset Source.....	5
2.2 Dataset Summary	5
2.3 Sentiment Labelling Rule.....	7
2.4 Text Preprocessing	7
3. Sentiment Model Development	7
3.1 Feature Extraction	7
3.2 Model Selection	7
3.3 Best Model	8
4. Apache System Architecture	10
4.1 Overall Architecture	10
4.2 Kafka Producer.....	10
4.3 Spark Structured Streaming	11
4.4 Elasticsearch Storage	11
4.5 Kibana Visualization	12
5. Analysis and Results.....	12
5.1 Data and Model Results	12
5.2 Streaming Pipeline Results	12
5.3 Kibana Dashboard Results	13
5.4 Key Findings	16
6. Optimization and Comparison	17
6.1 Batch Mode vs Streaming Mode.....	17
6.2 Model and Pipeline Improvements	17
6.3 Limitations	17
7. Conclusion and Future Work.....	18

Appendix..... 19

Appendix A: Project Folder Structure..... 19

Appendix B: Pipeline Evidence Screenshots 20

Appendix C: Kibana Dashboard Evidence 22

1. Introduction

1.1 Background

Telecommunication services are used daily by Malaysian customers for mobile data, calls, payments, reloads, and account management. Because many users leave comments and ratings on Google Play, these reviews provide a practical source of customer feedback. However, reading thousands of reviews manually is slow and inconsistent. A real-time sentiment analysis pipeline can help transform raw text reviews into structured sentiment insights that are easier to monitor and explain.

This project focuses on building a complete data processing pipeline rather than only creating a standalone sentiment model. The final workflow combines natural language processing, machine learning, Kafka message streaming, Spark structured processing, Elasticsearch storage, and Kibana visualization. In simple terms, the model acts as the brain, Kafka and Spark act as the data pipeline, and Kibana acts as the dashboard for understanding the results.

1.2 Problem Statement

Malaysian telecommunication app reviews are large, unstructured, and continuously generated. Without automated processing, it is difficult to identify whether customer feedback is mostly positive, neutral, or negative, and whether certain providers receive more complaints than others. Therefore, a scalable pipeline is required to ingest review data, process it in near real time, classify sentiment, store the classified results, and present the outcome through a visual dashboard.

1.3 Objectives

Table 1. Project objectives and implementation evidence.

No.	Objective	Implementation Evidence
1	Clean and prepare Malaysian telecommunication review text for sentiment analysis.	preprocessing.ipynb and cleaned_data.csv
2	Train and compare at least two sentiment classification models.	Naive Bayes and Logistic Regression in model_training.ipynb
3	Implement a streaming pipeline using Apache Kafka and Apache Spark.	kafka_producer.py and spark_streaming.py
4	Store classified sentiment output in Elasticsearch.	telecom-sentiment index and elastic_mappings.json
5	Visualize sentiment insights using Kibana.	Kibana dashboard screenshots and kibana_visualizations.ndjson

6	Compare batch model development with streaming processing performance.	Model accuracy, producer throughput, and Spark streaming throughput
---	--	---

1.4 Scope

The project scope covers Malaysian telecommunication Google Play app reviews from four providers: Celcom, U Mobile, Maxis, and Digi. The review rating score is used to generate sentiment labels, and the review text is used as the input feature for model training and streaming classification. The implemented system is a local development and demonstration pipeline, not a production deployment. Therefore, the streaming test was conducted using a subset of the cleaned dataset to prove the end-to-end flow.

2. Data Acquisition and Preprocessing

2.1 Dataset Source

The project uses the Malaysian Telecommunication Google Play reviews dataset. The raw dataset contains customer reviews, ratings, helpful-vote counts, review dates, and provider names. The raw file contains 159,444 rows and 6 columns. After cleaning, the final dataset contains 155,902 rows.

Table 2. Dataset fields used in the project.

Field	Description	Usage in Project
userName	Reviewer name stored in the source dataset.	Not used for modelling decisions; retained only as source metadata.
content	Original review text.	Main raw text input before cleaning.
score	Google Play rating score.	Converted into Positive, Neutral, or Negative labels.
thumbsUpCount	Number of helpful votes.	Available metadata; not used in the main model.
at	Review date.	Used for review trend analysis in Kibana.
provider	Telco provider name.	Used for provider-level sentiment comparison.

2.2 Dataset Summary

Table 3. Cleaned dataset summary.

Item	Value
Raw dataset rows	159,444
Cleaned dataset rows	155,902
Rows removed during cleaning	3,542 (2.22%)

Date range	2018-01-01 to 2023-12-04
Providers	Celcom, U Mobile, Maxis, Digi
Negative reviews in cleaned data	73,333 (47.04%)
Neutral reviews in cleaned data	9,248 (5.93%)
Positive reviews in cleaned data	73,321 (47.03%)

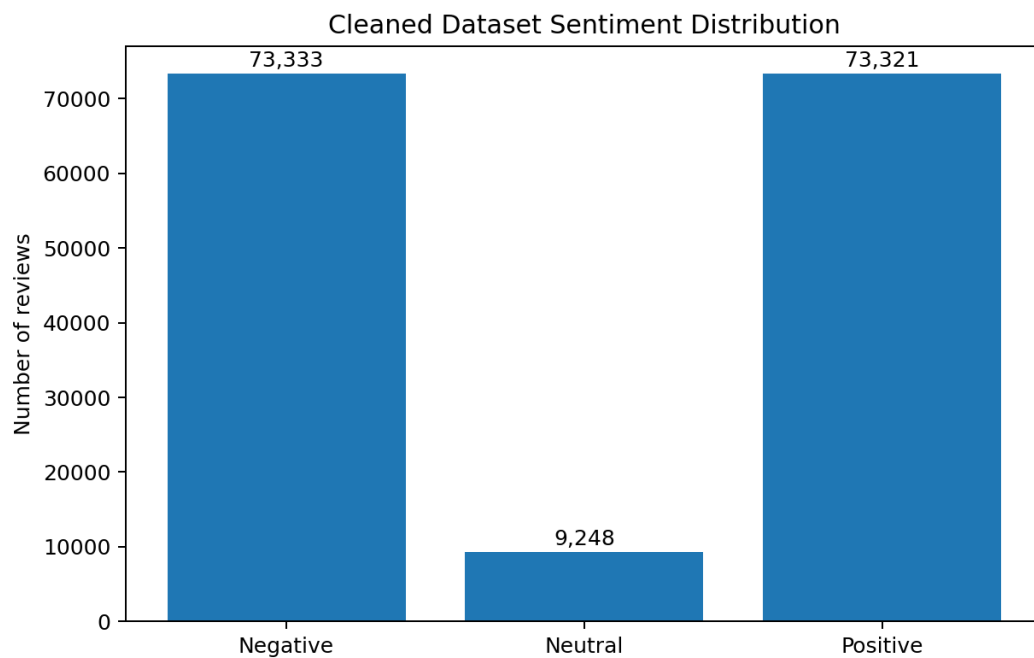


Figure 2. Cleaned dataset sentiment distribution.

Figure 2 shows the sentiment distribution of the cleaned dataset after preprocessing and rating-based labelling. The dataset contains 73,333 negative reviews (47.04%), 73,321 positive reviews (47.03%), and 9,248 neutral reviews (5.93%). This shows that the negative and positive classes are almost balanced, while the neutral class is much smaller.

2.3 Sentiment Labelling Rule

Table 4. Sentiment labelling rule.

Rating Score	Sentiment Label	Reason
1-2	Negative	Low rating normally represents dissatisfaction or complaints.
3	Neutral	Middle rating normally represents mixed or average feedback.
4-5	Positive	High rating normally represents satisfaction or approval.

2.4 Text Preprocessing

The raw review text was cleaned before vectorization and modelling. The preprocessing notebook performs the following major operations:

- Converted text to lowercase for consistent matching.
- Removed URLs, mentions, hashtags, punctuation, emojis, and special symbols.
- Kept useful letters, numbers, and spaces for model input.
- Collapsed repeated whitespace and removed empty cleaned reviews.
- Exported the final cleaned dataset as data/cleaned_data.csv.

3. Sentiment Model Development

3.1 Feature Extraction

The cleaned review text was converted into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF). A maximum of 5,000 features was used to keep the model lightweight enough for integration into the streaming pipeline. This choice helps the model run quickly when Spark processes incoming micro-batches.

3.2 Model Selection

Two supervised machine learning models were trained and compared: Multinomial Naive Bayes and Logistic Regression. Naive Bayes was selected as a simple baseline model, while Logistic Regression was selected because it often performs well on sparse TF-IDF text features.

Table 5. Model comparison result.

Model	Accuracy	Weighted F1-score	Macro F1-score	Decision
Naive Bayes	83.06%	80.56%	57.24%	Baseline model
Logistic Regression	86.46%	83.95%	60.05%	Selected model

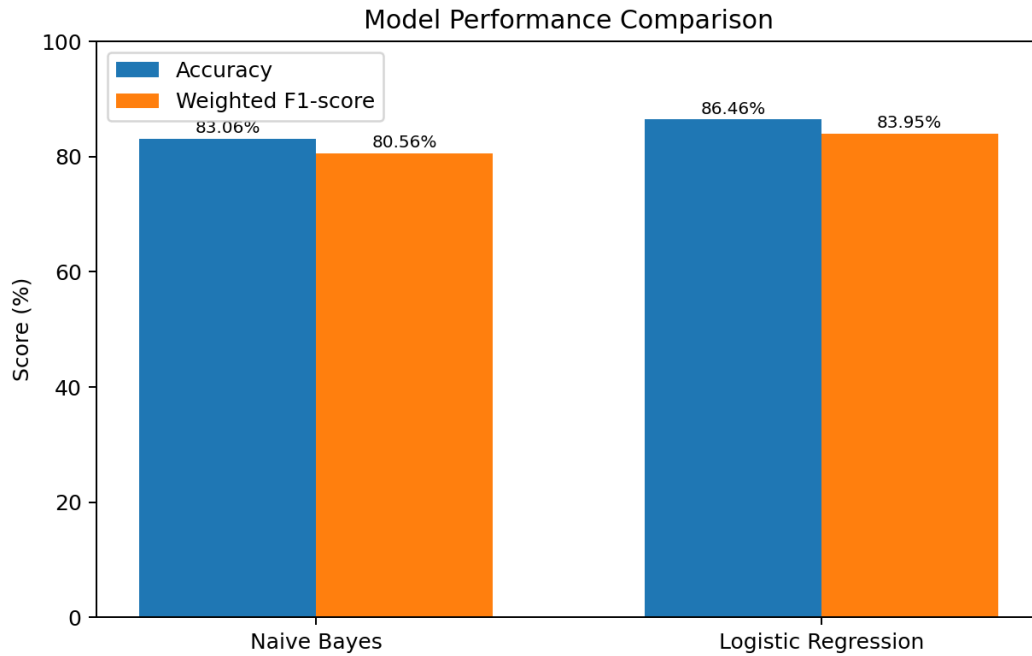


Figure 3. Model performance comparison between Naive Bayes and Logistic Regression.

Figure 3 shows the comparison between the two trained sentiment models. Logistic Regression achieved the higher accuracy at 86.46%, compared with 83.06% for Naive Bayes. It also achieved a higher weighted F1-score of 83.95%, compared with 80.56% for Naive Bayes, which supports the selection of Logistic Regression as the final model.

3.3 Best Model

Logistic Regression achieved the higher accuracy and weighted F1-score, so it was selected as the final deployed sentiment model. The selected model and vectorizer were saved as pickle files and passed to the streaming pipeline.

```
models/sentiment_model.pkl # trained Logistic Regression classifier
models/vectorizer.pkl      # TF-IDF vectorizer used before prediction
```

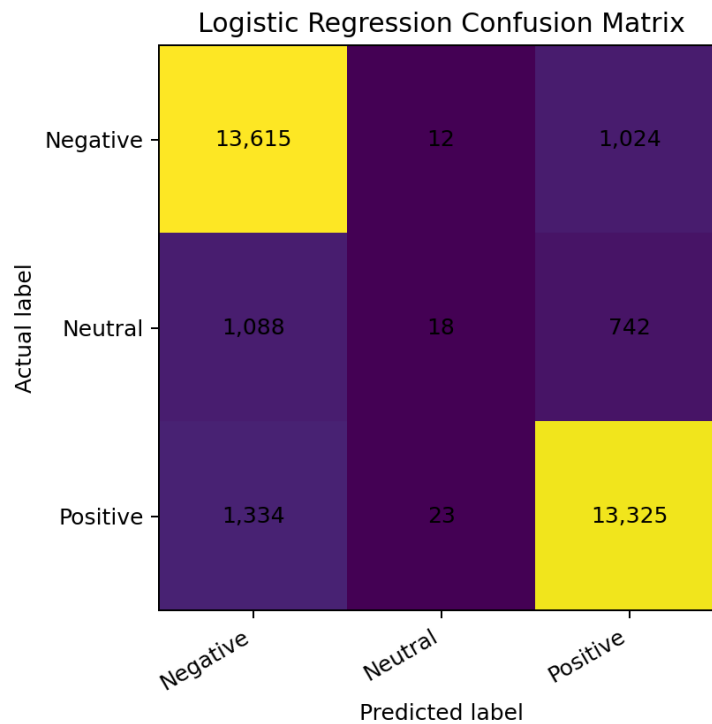


Figure 4. Logistic Regression confusion matrix.

Figure 4 shows the confusion matrix for the selected Logistic Regression model. The model correctly classified 13,615 negative reviews and 13,325 positive reviews, showing strong performance for the two dominant sentiment classes. However, only 18 neutral reviews were correctly classified, which indicates that neutral sentiment was harder to detect because the neutral class was much smaller and often contained mixed wording.

4. Apache System Architecture

4.1 Overall Architecture

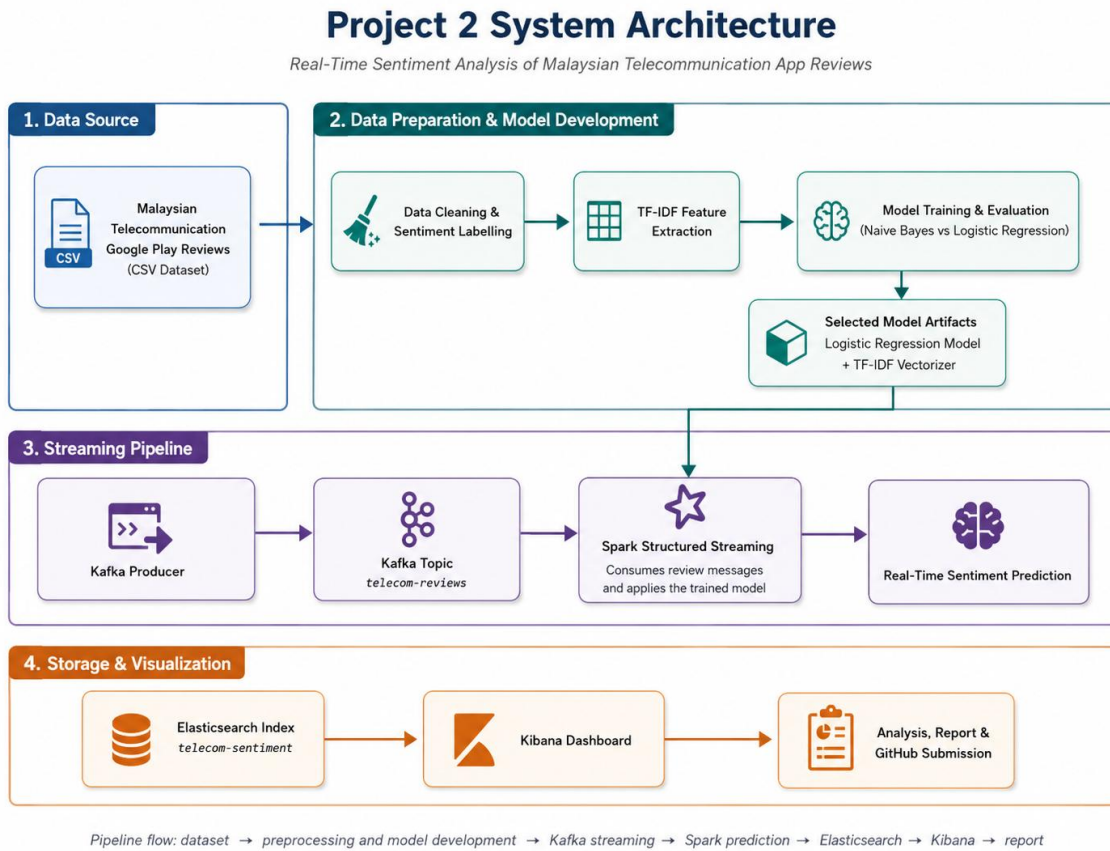


Figure 1. Overall system architecture for the real-time sentiment analysis pipeline.

Figure 1 shows the complete architecture of the real-time sentiment analysis system. The workflow starts from the Malaysian Telecommunication Google Play Reviews dataset, continues through data cleaning, TF-IDF feature extraction, and model training, then moves into the Kafka and Spark streaming pipeline. The selected Logistic Regression model and TF-IDF vectorizer are loaded into Spark Structured Streaming so that incoming review messages can be classified in near real time. The predicted results are stored in the telecom-sentiment Elasticsearch index and visualized through Kibana dashboards for sentiment distribution, provider comparison, review trends, rating patterns, and negative feedback analysis.

4.2 Kafka Producer

The Kafka producer reads `data/cleaned_data.csv` and sends one review per message. Each message contains the cleaned review text, score, provider, review date, and original rating-based sentiment label. A small delay is used between messages to simulate continuous review arrival.

```
{
  "userName": "...",
  "cleaned_content": "...",
  "score": 1,
  "provider": "maxis",
  "review_date": "2023-07-02",
  "original_sentiment": "Negative"
}
```

4.3 Spark Structured Streaming

Spark Structured Streaming consumes the telecom-reviews Kafka topic and processes records using `foreachBatch`. In each micro-batch, the cleaned text is transformed using the saved TF-IDF vectorizer and classified using the saved Logistic Regression model. The output includes predicted sentiment, the original sentiment label, and a match flag for validation.

```
[Batch 751] Processing 12 records...
[Batch 751] ✓ Done – 12 records in 0.34s (35 rec/s) | Cumulative: 11,831 records, 28 rec/s
[Batch 751] Sample predictions:
  userName provider original_sentiment predicted_sentiment match
A Google user maxis Positive Positive True
A Google user maxis Positive Positive True
ely helena maxis Negative Negative True

[Batch 752] Processing 10 records...
[Batch 752] ✓ Done – 10 records in 0.35s (29 rec/s) | Cumulative: 11,841 records, 28 rec/s
[Batch 752] Sample predictions:
  userName provider original_sentiment predicted_sentiment match
Yuvannesh Mayadeven maxis Positive Positive True
Ummun Nadzirah maxis Negative Positive False
Daniel Luqman maxis Positive Positive True
```

Figure 6. Spark sample predictions produced by the streaming pipeline.

Figure 6 shows sample prediction records produced by the Spark Structured Streaming pipeline. Each incoming Kafka message is transformed using the saved TF-IDF vectorizer, classified using the Logistic Regression model, and enriched with predicted sentiment and match status before being written to Elasticsearch. This figure provides evidence that the streaming pipeline successfully applied the trained model to live review messages.

4.4 Elasticsearch Storage

Elasticsearch stores the prediction output in the telecom-sentiment index. This index supports fast filtering and aggregation by provider, sentiment, score, date, and match status. Kibana uses this index as its data source.

Table 6. Elasticsearch index field mapping.

Field	Type	Purpose
provider	keyword	Provider-level filtering and aggregation.
cleaned_content	text	Searchable cleaned review text.
score	integer	Rating distribution analysis.
predicted_sentiment	keyword	Sentiment distribution and provider comparison.
review_date	date	Trend analysis over time.
match	boolean	Validation indicator showing whether prediction matched the original label.

4.5 Kibana Visualization

Kibana was used to create dashboard panels from the Elasticsearch sentiment index. The dashboard includes sentiment distribution, provider breakdown, correct prediction count, review trend, rating distribution, top negative providers, and common negative review terms.

5. Analysis and Results

5.1 Data and Model Results

The preprocessing phase reduced the raw dataset from 159,444 records to 155,902 cleaned records. The final cleaned dataset contains 73,333 negative reviews, 9,248 neutral reviews, and 73,321 positive reviews. The model development result shows that Logistic Regression outperformed Naive Bayes and was therefore selected for the streaming deployment.

5.2 Streaming Pipeline Results

The pipeline was tested by streaming a subset of the cleaned dataset. The Kafka producer successfully sent review messages to the telecom-reviews topic, and Spark processed the incoming micro-batches before writing the output to Elasticsearch.



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS
elasticsearch	docker.elastic.co/elasticsearch/elasticsearch:8.14.0	"/bin/tini -- /usr/l..."	elasticsearch	About a minute ago	Up About a minute (he
kafka	confluentinc/cp-kafka:7.7.0	"/etc/confluent/dock..."	kafka	About a minute ago	Up About a minute (he
kibana	docker.elastic.co/kibana/kibana:8.14.0	"/bin/tini -- /usr/l..."	kibana	About a minute ago	Up 36 seconds
zookeeper	confluentinc/cp-zookeeper:7.7.0	"/etc/confluent/dock..."	zookeeper	About a minute ago	Up About a minute (he
spark	2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp, 3888/tcp				

Figure 5. Docker infrastructure running for Kafka, Spark, Elasticsearch, and Kibana.

Figure 5 shows the Docker-based infrastructure used to run the streaming pipeline. The running containers include Kafka, Zookeeper, Elasticsearch, Kibana, the Kafka producer, and the Spark

streaming service. This confirms that the required services were active together during the end-to-end streaming test.

Table 7. Streaming test performance summary.

Metric	Observed Value
Producer messages sent	17,633 messages
Producer throughput	19.5 records/sec
Spark records processed	17,251 records
Spark micro-batches processed	1,176 batches
Spark throughput	28.1 records/sec
Elasticsearch index	telecom-sentiment

The number of processed Spark records is slightly lower than the number of produced Kafka messages because the test run was stopped during local execution. This does not affect the proof of concept, because the pipeline still demonstrates the full flow from Kafka input to Spark prediction and Elasticsearch storage.

5.3 Kibana Dashboard Results

The Kibana dashboard was used mainly as a validation dashboard for correctly predicted streaming records. Therefore, the dashboard percentages reported below should be interpreted as the distribution within the correctly predicted records available in the dashboard evidence, rather than as a complete summary of the entire 155,902-row cleaned dataset.

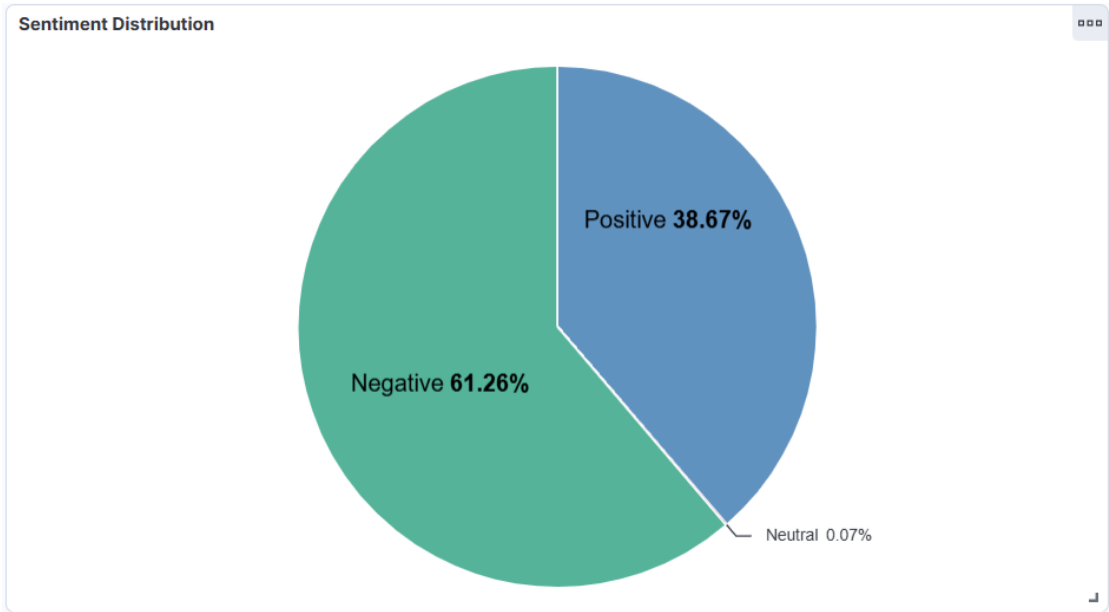


Figure 7. Kibana sentiment distribution for correctly predicted streaming records.

Figure 7 shows the sentiment distribution for correctly predicted records in the Kibana validation dashboard. Negative sentiment accounts for 61.26% of the records, positive sentiment accounts for 38.67%, and neutral sentiment accounts for only 0.07%. This indicates that negative feedback is dominant within the correctly predicted streaming records shown in the dashboard.

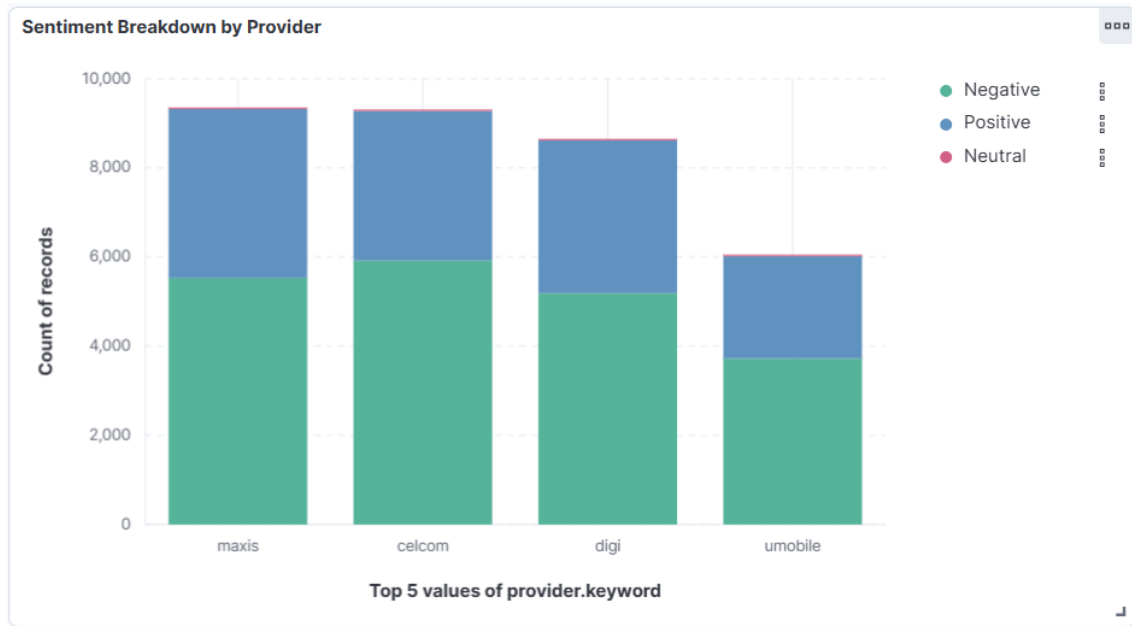


Figure 8. Sentiment breakdown by telecommunication provider.

Figure 8 shows the sentiment breakdown by provider for Maxis, Celcom, Digi, and U Mobile. The chart indicates that all four providers contain both positive and negative reviews, but negative sentiment remains a major portion across the providers. This means that customer dissatisfaction is not limited to a single provider, although the negative counts differ by provider.

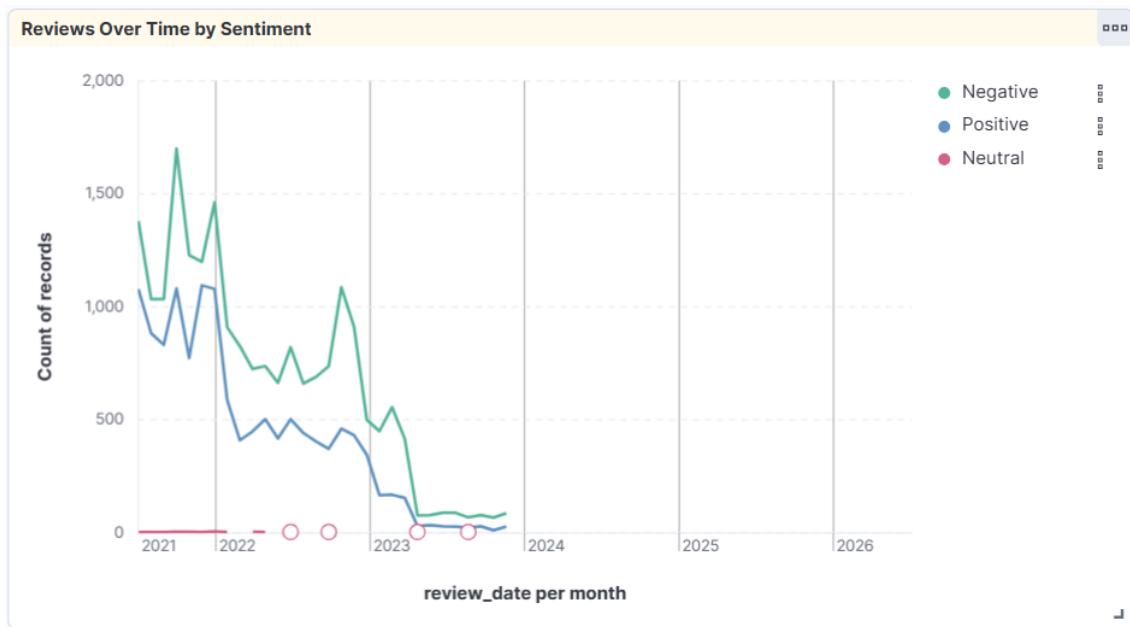


Figure 9. Review sentiment trend over time.

Figure 9 shows the monthly review sentiment trend over time. The review activity is mainly concentrated from 2021 to 2023, and the negative sentiment line remains higher than the positive sentiment line during much of the active period. This suggests that negative customer feedback was consistently visible across the main review period.

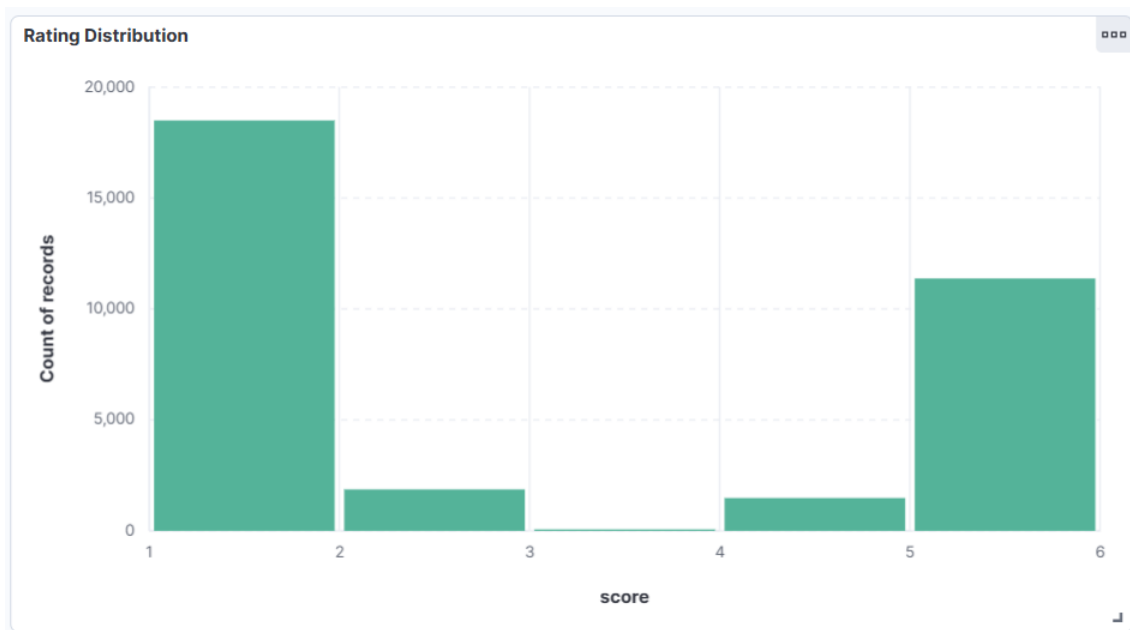
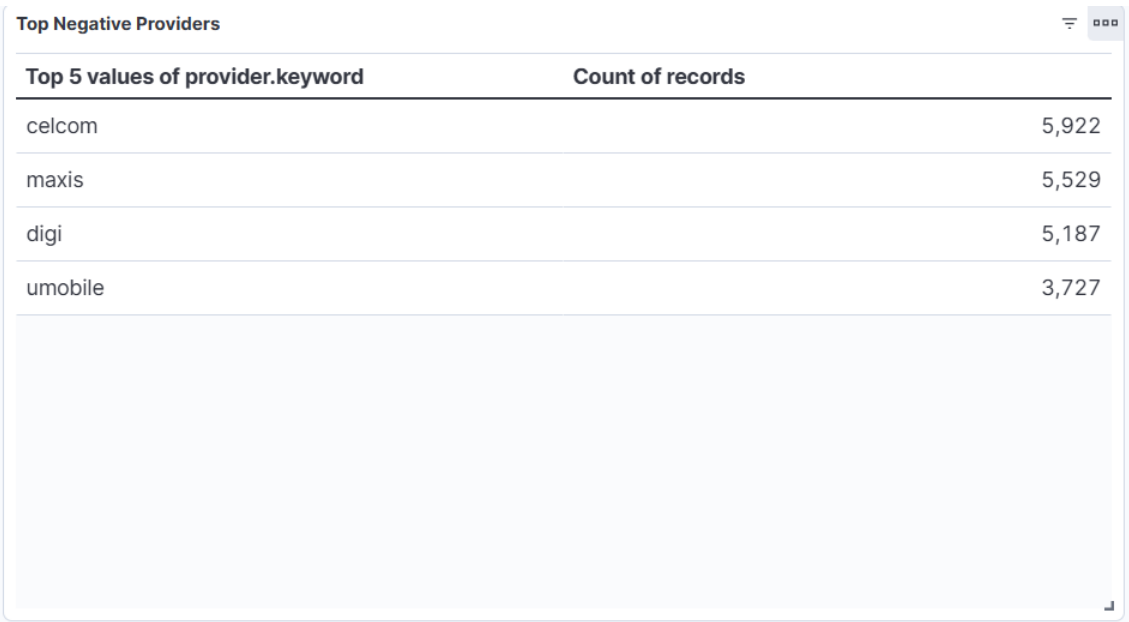


Figure 10. Rating distribution of streamed records.

Figure 10 shows the rating distribution of the streamed records used in the dashboard evidence. The largest group is 1-star reviews, followed by a large number of 5-star reviews. This pattern supports the

sentiment results because low ratings are mapped to negative sentiment, while high ratings are mapped to positive sentiment.



The screenshot shows a Kibana dashboard titled 'Top Negative Providers'. It contains a table with two columns: 'Top 5 values of provider.keyword' and 'Count of records'. The table lists four providers: celcom (5,922), maxis (5,529), digi (5,187), and umobile (3,727). The table is part of a larger visualization that includes a search bar and a filter icon in the top right corner.

Top 5 values of provider.keyword	Count of records
celcom	5,922
maxis	5,529
digi	5,187
umobile	3,727

Figure 11. Top negative providers from the Kibana dashboard.

Figure 11 shows the top negative providers based on the Kibana dashboard. Celcom has the highest negative count with 5,922 records, followed by Maxis with 5,529, Digi with 5,187, and U Mobile with 3,727. This ranking highlights where negative feedback is most concentrated in the dashboard evidence.

5.4 Key Findings

The cleaned dataset shows that negative and positive reviews are almost evenly distributed, while neutral reviews appear much less frequently. This indicates that most users tend to leave reviews when they have either a clearly positive or clearly negative experience, instead of giving moderate feedback. In the model comparison, Logistic Regression performed better than Naive Bayes when TF-IDF features were used, making it the selected model for the streaming pipeline.

The Kafka and Spark pipeline was able to stream review records, apply sentiment prediction, and store the processed output in Elasticsearch. Kibana then converted the Elasticsearch records into useful visual insights, including sentiment distribution, provider comparison, review trends, and rating distribution. From the negative review analysis, the word “slow” appeared repeatedly, suggesting that service speed, network performance, or app responsiveness may be common complaint areas among Malaysian telecommunication app users.

6. Optimization and Comparison

6.1 Batch Mode vs Streaming Mode

Table 8. Batch mode and streaming mode comparison.

Aspect	Batch Mode	Streaming Mode
Input	Full cleaned CSV dataset used in notebooks.	Kafka messages produced from cleaned CSV rows.
Processing method	TF-IDF vectorization and model training/testing in notebook.	Spark Structured Streaming micro-batches with saved model inference.
Main output	Model accuracy and saved model artifacts.	Prediction records written to Elasticsearch.
Observed result	Logistic Regression accuracy: 86.46%.	17,251 records processed at 28.1 records/sec.
Best use case	Historical analysis and model development.	Near real-time sentiment monitoring and dashboard updates.

Batch processing was useful for building and evaluating the sentiment model using the full cleaned dataset. Streaming processing was useful for demonstrating how new reviews can be processed continuously and stored for dashboard use. In this project, batch mode produced the final trained model, while streaming mode proved that the trained model could be integrated into a real-time data pipeline.

6.2 Model and Pipeline Improvements

- The neutral class can be improved by collecting more neutral examples or using class weighting.
- The Spark pipeline can avoid `toPandas()` for larger production workloads by using a Spark-native model or UDF strategy.
- Kafka starting offsets can be adjusted during testing to avoid missing messages when Spark starts after the producer.
- The scikit-learn version used for model training and Docker runtime should be aligned to reduce pickle compatibility warnings.
- The Kibana dashboard can be separated into two views: one for all predictions and one for validation records where match is true.

6.3 Limitations

The implementation was completed as a local demonstration system. Because of local runtime and hardware constraints, only a subset of the cleaned dataset was streamed during the recorded test run. In addition, the dashboard screenshots mainly show correctly predicted records because the exported

dashboard includes a `match:true` filter. These limitations do not remove the value of the system, but they should be considered when interpreting the dashboard percentages.

A Spark/Py4J connection warning was observed near the end of one streaming screenshot. The final streaming summary was still produced and records had already been processed before termination, so the issue is treated as a runtime shutdown or local execution limitation rather than a failure of the whole pipeline.

7. Conclusion and Future Work

This project successfully demonstrates an end-to-end real-time sentiment analysis pipeline for Malaysian telecommunication app reviews. The raw review dataset was cleaned, labelled, and prepared for sentiment classification. Two machine learning models, Naive Bayes and Logistic Regression, were trained and compared using TF-IDF features, and Logistic Regression was selected as the final model because it achieved better performance. The selected model was then integrated into a Kafka and Spark Structured Streaming pipeline, where review messages were streamed, classified, and stored in Elasticsearch.

Kibana was used to visualize the stored sentiment results through charts and tables. The dashboard provided useful insights into sentiment distribution, provider-level comparison, review trends, rating patterns, and negative feedback terms. Based on the validation dashboard, negative feedback appeared to be dominant, with Celcom recording the highest negative count among the providers shown. This shows that the pipeline was not only able to process data in real time, but also able to convert the processed output into meaningful analytical insights.

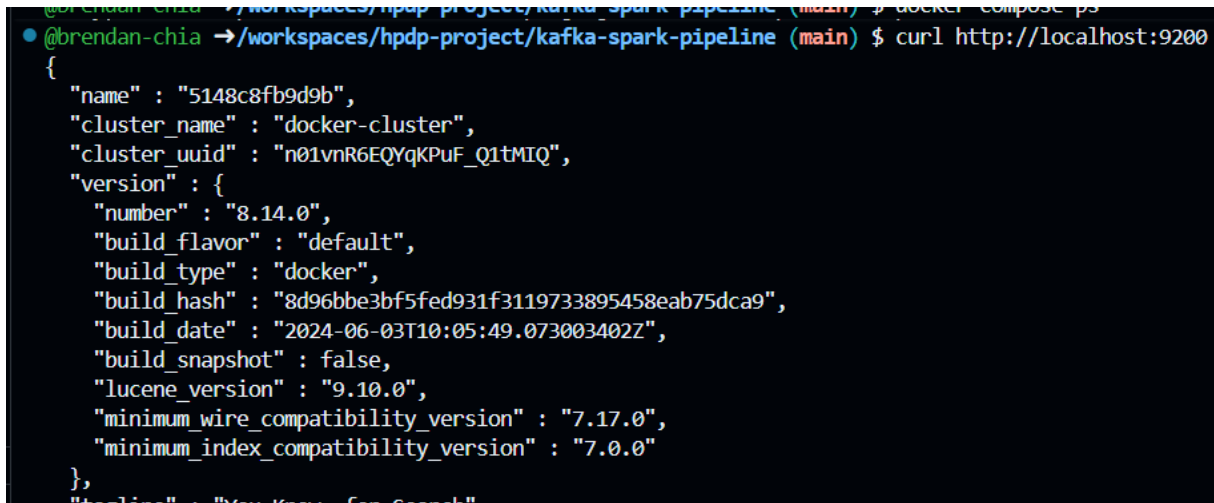
Overall, the project meets the main technical requirements by combining data preprocessing, machine learning, Kafka streaming, Spark processing, Elasticsearch storage, and Kibana visualization in one complete workflow. Future improvements can include using a transformer-based sentiment model, streaming a larger portion of the dataset over a longer test period, creating a full unfiltered Kibana dashboard, and deploying the system on a cloud or cluster environment for stronger scalability testing.

Appendix

Appendix A: Project Folder Structure

```
DataAnalysis/
├── dashboard/
│   ├── 01_sentiment_distribution.png
│   ├── 02_sentiment_breakdown_by_provider.png
│   ├── 03_correct_prediction_records.png
│   ├── 04_reviews_over_time.png
│   ├── 05_top_negative_providers.png
│   ├── 06_rating_distribution.png
│   ├── 07_dashboard_overview.png
│   ├── elastic_mappings.json
│   ├── kibana_visualizations.ndjson
│   └── visual_analysis.md
├── data/
│   ├── cleaned_data.csv
│   └── raw_data/
├── kafka-spark-pipeline/
│   ├── config.py
│   ├── docker-compose.yml
│   ├── Dockerfile
│   ├── kafka_producer.py
│   ├── README.md
│   ├── requirements.txt
│   └── spark_streaming.py
├── models/
│   ├── sentiment_model.pkl
│   └── vectorizer.pkl
├── notebooks/
│   ├── preprocessing.ipynb
│   └── model_training.ipynb
├── reports/
└── screenshots member2/
```

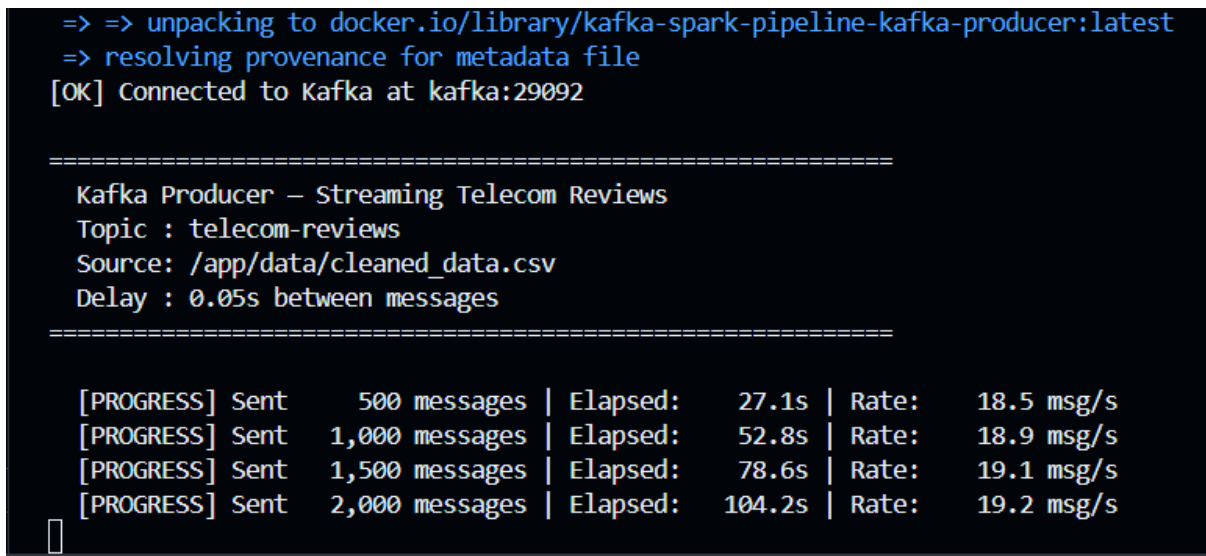
Appendix B: Pipeline Evidence Screenshots



```
@brendan chia → /workspaces/hdp-project/kafka-spark-pipeline (main) $ docker compose ps
● @brendan chia → /workspaces/hdp-project/kafka-spark-pipeline (main) $ curl http://localhost:9200
{
  "name" : "5148c8fb9d9b",
  "cluster_name" : "docker-cluster",
  "cluster_uuid" : "n01vnR6EQYqKPuF_Q1tMIQ",
  "version" : {
    "number" : "8.14.0",
    "build_flavor" : "default",
    "build_type" : "docker",
    "build_hash" : "8d96bbe3bf5fed931f3119733895458eab75dca9",
    "build_date" : "2024-06-03T10:05:49.073003402Z",
    "build_snapshot" : false,
    "lucene_version" : "9.10.0",
    "minimum_wire_compatibility_version" : "7.17.0",
    "minimum_index_compatibility_version" : "7.0.0"
  },
  "tagline" : "You Know, for Search"
```

Appendix Figure B1. Elasticsearch responding at localhost:9200.

Appendix Figure B1 shows that Elasticsearch was responding successfully at localhost:9200. This confirms that the storage service was active and ready to receive prediction records from the Spark streaming pipeline.



```
=> => unpacking to docker.io/library/kafka-spark-pipeline-kafka-producer:latest
=> resolving provenance for metadata file
[OK] Connected to Kafka at kafka:29092

=====
Kafka Producer – Streaming Telecom Reviews
Topic : telecom-reviews
Source: /app/data/cleaned_data.csv
Delay : 0.05s between messages
=====

[PROGRESS] Sent      500 messages | Elapsed:   27.1s | Rate:    18.5 msg/s
[PROGRESS] Sent    1,000 messages | Elapsed:   52.8s | Rate:    18.9 msg/s
[PROGRESS] Sent    1,500 messages | Elapsed:   78.6s | Rate:    19.1 msg/s
[PROGRESS] Sent    2,000 messages | Elapsed:  104.2s | Rate:    19.2 msg/s
█
```

Appendix Figure B2. Kafka producer streaming review messages.

Appendix Figure B2 shows the Kafka producer sending review messages into the telecom-reviews topic. The producer reads cleaned review rows and streams them as JSON messages, which proves that the Kafka ingestion stage was functioning during the test run.

```
=====
PRODUCER SUMMARY
=====
Total records sent : 17,633
Skipped (empty)    : 0
Total time         : 903.67 seconds
Throughput         : 19.5 records/sec
Finished at        : 2026-06-30 08:53:21
=====

Log saved to: /app/logs/producer_log.txt

[DONE] Producer finished. 17,633 messages sent to 'telecom-reviews'.
@brendan-chia → /workspaces/hdp-project/kafka-spark-pipeline (main) $
```

Appendix Figure B3. Kafka producer final summary.

Appendix Figure B3 shows the final producer summary, where 17,633 review messages were sent with an observed throughput of 19.5 records per second. This result provides evidence of the streaming input volume generated during the local test run.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
@brendan-chia → /workspaces/hdp-project/kafka-spark-pipeline (main) $ docker compose run spark-streaming
cable
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(newLevel).
[OK] SparkSession created.
[OK] Subscribed to Kafka topic.
26/06/30 08:38:36 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not supported in streaming DataFrames/Datasets and will be disabled.
[OK] Streaming query started. Waiting for messages...

Press Ctrl+C to stop the pipeline.

26/06/30 08:38:37 WARN AdminClientConfig: These configurations '[key.deserializer, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]' were supplied but are not used yet.

[Batch 0] Processing 3 records...
[Batch 0] ✓ Done – 3 records in 9.72s (0 rec/s) | Cumulative: 3 records, 0 rec/s
[Batch 0] Sample predictions:
  userName provider original_sentiment predicted_sentiment match
```

Appendix Figure B4. Spark streaming micro-batch processing.

Appendix Figure B4 shows Spark Structured Streaming processing incoming Kafka messages in micro-batches. The screenshot provides runtime evidence that Spark consumed the streamed records, applied prediction logic, and continued processing batches during the experiment.

```
raise Py4JNetworkError(
py4j.protocol.Py4JNetworkError: Error while sending or receiving

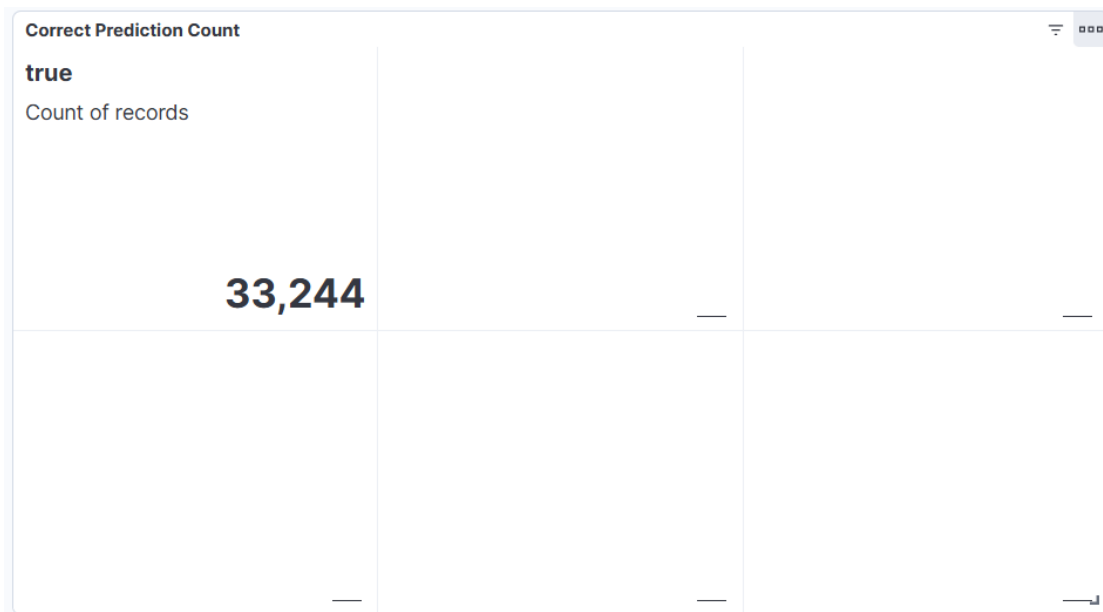
=====
STREAMING PIPELINE SUMMARY
=====
Total records processed : 17,251
Total batches           : 1176
Total processing time   : 613.70s
Overall throughput      : 28.1 records/sec
Pipeline uptime         : 950.44s
Elasticsearch index     : telecom-sentiment
Finished at             : 2026-06-30 08:53:59
=====

Log saved to: /app/logs/streaming_log.txt
```

Appendix Figure B5. Spark streaming final summary.

Appendix Figure B5 shows the final Spark streaming summary. The pipeline processed 17,251 records across 1,176 micro-batches with an observed throughput of 28.1 records per second, showing that the streaming stage successfully processed a subset of the cleaned dataset.

Appendix C: Kibana Dashboard Evidence



Appendix Figure C1. Correct prediction count shown in Kibana.

Appendix Figure C1 shows the Kibana validation metric for correct prediction records. The dashboard reports 33,244 records where the predicted sentiment matched the original rating-based sentiment label, which supports the use of the match field as a validation indicator.

Top 5 values of predicted_sentime	Top 3 values of provider.keyword	Top 3 values of cleaned_content.k	review_date per month	Count of records
Negative	celcom	slow	2021-07-02	9
Negative	celcom	slow	2021-08-01	4
Negative	celcom	slow	2021-08-31	3
Negative	celcom	slow	2021-09-30	4
Negative	celcom	slow	2021-10-30	5
Negative	celcom	slow	2021-11-29	2
Negative	celcom	slow	2021-12-29	9
Negative	celcom	slow	2022-01-28	4
Negative	celcom	slow	2022-02-27	2

Appendix Figure C2. Common negative review terms over time.

Appendix Figure C2 shows common negative review terms over time. The term slow appears repeatedly, especially in the negative-review summaries, suggesting that service speed, network performance, or app responsiveness is a frequent complaint area in the analysed reviews.

The Kibana dashboard export file kibana_visualizations.ndjson and Elasticsearch mapping file elastic_mappings.json are included in the dashboard folder for submission evidence.